From Data to Solutions · Weekend 7

SLURM: getting a job onto a cluster

How a shared machine decides whose job runs next

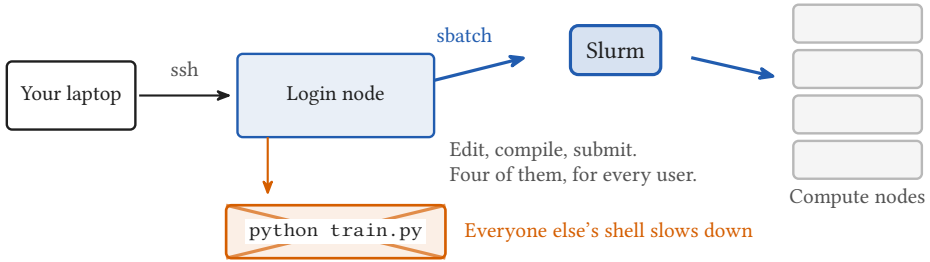
Carlos Cotrini · Presented by Ghali

Section 1

The shared machine

1

The login node is not the cluster



- Not a bigger laptop: a shared machine with an allocation policy
- Only Slurm puts work on a compute node, and `sbatch` is how you ask
- Running the training on the login node is the classic first mistake

Three partitions, four states

Three partitions on this machine, each with its own limits

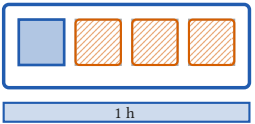
debug	**normal**, the default	**xfer**
32 nodes	1266 nodes	2 nodes
1 to 2 nodes per job, 30 min	No node limit, 24 h	1 node per job, 24 h

Node sharing is not available on Alps: every job gets whole nodes.

One sbatch script, nine lines

`#!/bin/bash`	The shell that runs it
`#SBATCH --account=<project>`	Which budget pays
`#SBATCH --partition=normal`	Which pool of nodes
`#SBATCH --nodes=1`	How many nodes
`#SBATCH --ntasks-per-node=1`	One process on the node
`#SBATCH --gpus-per-task=1`	One GPU for that process
`#SBATCH --time=01:00:00`	Stopped after one hour
`#SBATCH --output=logs/%x-%j.log`	Where the output lands
`srun python train.py`	Launch it on the allocation

One node, four GPUs



Three of them idle

Alps has no node sharing, so this job holds the whole node: four GPUs and 288 cores, one of them used.

Section 2

Why you are waiting

2

Why you are waiting

Four factors set your place in the queue

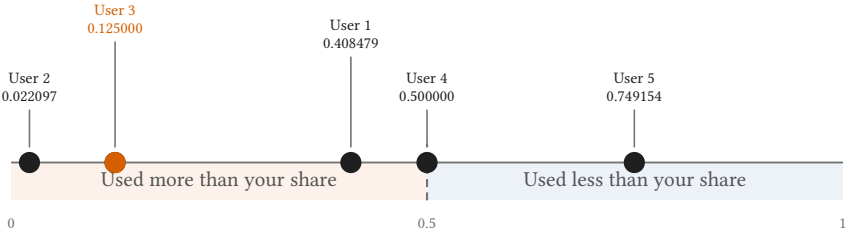
Job priority: one integer, 0 to 4,294,967,295, recomputed every five minutes

And one that is not: ~~Requested wall time~~ Nowhere in the formula

The weight on each factor is a site setting. Run `scontrol show config` to see what this machine actually does.

Why you are waiting

Fairshare: 0.5 is exactly your share



User 3 has run nothing. That number is their account-mates' usage.

Fairshare is charged to the account, not to you, and usage decays with a seven day half life by default (`PriorityDecayHalfLife`).

Section 3

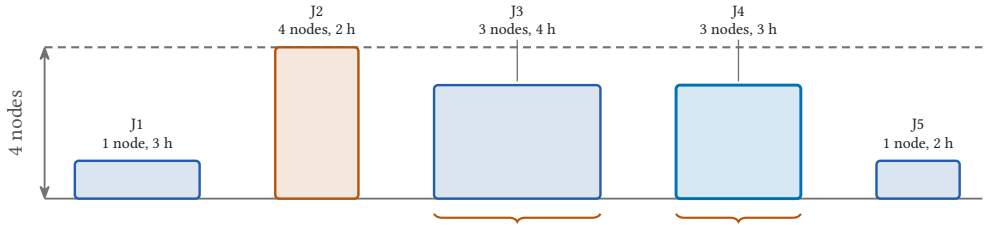
Backfill

3

Backfill

Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide

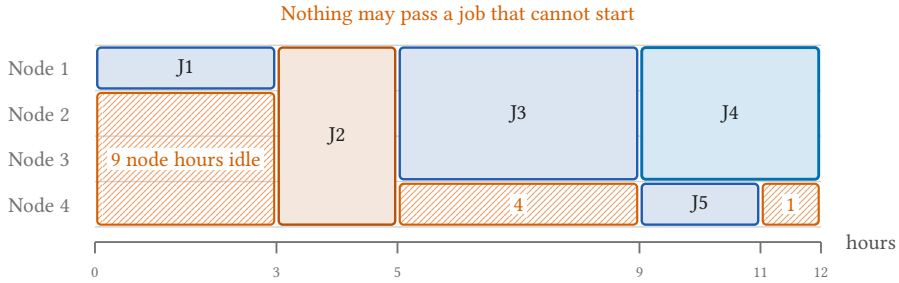


The area is what you are charged: 34 node hours in all

J3 and J4 are the same three nodes. Only the time asked for differs.

All five are submitted at the same moment, and they are listed in priority order.

Strict priority: twelve hours

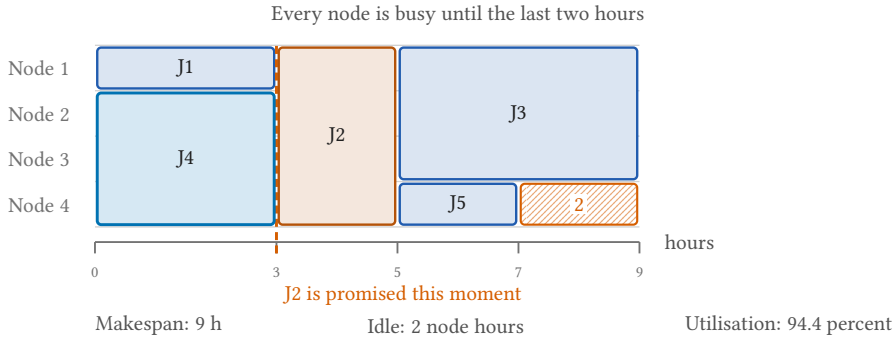


Makespan: 12 h

Idle: 14 node hours

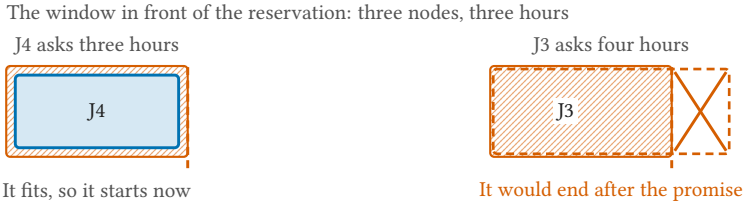
Utilisation: 70.8 percent

Backfill: the same five jobs, nine hours



Nodes held for that promise look idle. Slurm calls them `planned`.

A shorter request fits more holes



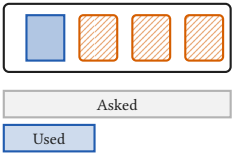
- J3 is ahead of J4 in the queue the whole time, and starts five hours later
- A shorter request does not promote you: it makes you fit
- Had J3 asked for the three hours it needs, it would take the window
- Ask too little and Slurm stops your job at the limit you gave

Section 4

What wastes an allocation

4

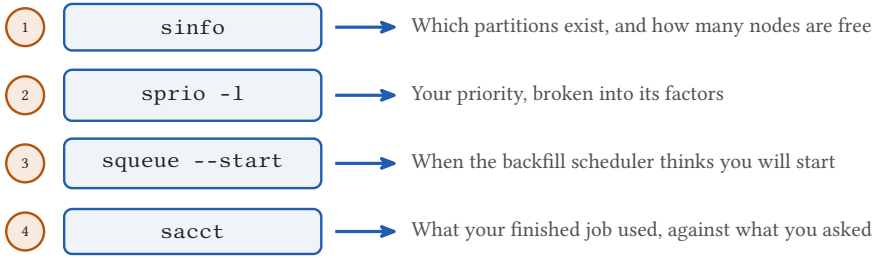
Four ways to waste an allocation



One GPU, a whole node	Four GPUs billed, one used
Wall time far above what you need	Costs queue position only
A thousand separate submissions	Use one `--array` job
Training on the login node	Four of them, for every user

Nodes and GPUs are charged whether you use them. Wall time is not: Slurm charges the time your job actually ran.

Four commands, and what each one shows



- The circled number is the section of this deck each command makes visible
- At 14:00 you write one of these scripts and put it into the queue